Parallel Computing, 2026SS

OpenMP - Part II

Outline

- **Data Dependencies**
  identifying and avoiding data hazards

- **Synchronization Constructs**
  coordination of threads and data access in parallel regions

- **Work-sharing constructs**
  divide the execution of the enclosed code region among the members of the team (e.g.: loop construct, sections/section constructs, single construct)

Last Class Recap

Spawns a team of threads at the fork

- Team = Master (ID = 0) + Workers (ID > 0)

Each member of the team executes the same code found within this parallel region

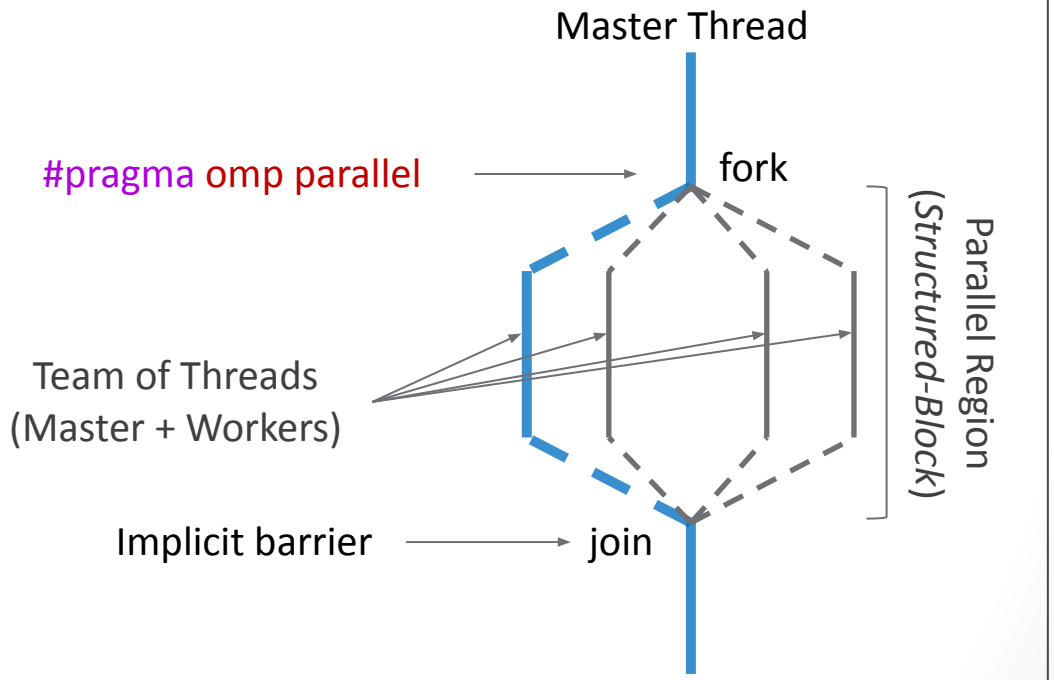
Implicit barrier at the end of the parallel region

- All threads must reach the barrier before proceeding

Variables can have different data-scoping:

- `private` **- only accessed by owner thread**
- `shared` **- all threads have access**

```
#pragma omp parallel [clauses]
{
  Structured-block
}
```



Program Order and Parallelism

In a sequential program the execution of its statements (instructions) presents a linear order:

- this order might be more strict than necessary,
- reordering of statements might be possible.

The goal of parallel programming is to reorder these statements in a way that:

- allows statements to be simultaneously executed,
- while ensuring sequential consistency (or close).

Dependence analysis is required to identify parallelism opportunities and ensure the correctness of a parallel program:

- Not done by OpenMP or compiler.

Sequential program:

```
Statement 1: a = x;
Statement 2: a++;
Statement 3: b = y;
Statement 4: b++;
Result: a = x+1 and b=y+1
```

Valid reordering:

```
Statement 3: b = y;
Statement 1: a = x;
Statement 4: b++;
Statement 2: a++;
Result: a = x+1 and b=y+1
```

Data Dependencies

There is a data dependence from `statement S1` to `statement S2` if:

1. S1 and S2 access the same location, and at least one of the accesses is a write,
2. and there is an execution path from S1 to S2.

True-dependence (Read-after-Write) - `statement S1` writes to a location that will be read by `S2`:

- **write by `S1` must happen before read by `S2`:**

```
S0: a = x;
S1: a = y;
S2: b = a + 1;
  b = y + 1;
```

```
S0: a = x;
S2: b = a + 1;
S1: a = y;
 b = x + 1;
```

Data Dependencies

Anti-dependence (Write-after-Read) - `statement S1` **reads from a location into which** `S2` **writes:**

- **read by** `S1` **must happen before write by** `S2`:

```
S0: b = x;
S1: a = b + 1;
S2: b = y;
```

```
  a = x + 1;
    b = y;
```

```
S0: b = x;
S2: b = y;
S1: a = b + 1;
```

```
  a = y + 1;
    b = y;
```

Variable Renaming:

```
S0: %b = x;
S2: b = y;
S1: a = %b + 1;
```

```
  a = x + 1;
    b = y;
```

Data Dependencies

Anti-dependence (Write-after-Read) - `statement S1` **reads from a location into which** `S2` **writes:**

- **read by** `S1` **must happen before write by** `S2`:

```
S0: b = x;
S1: a = b + 1;
S2: b = y;
-----------
  a = x + 1;
```

```
S0: b = x;
S2: b = y;
S1: a = b + 1;
-----------
 a = y + 1;
```

Variable Renaming:

```
S0: %b = x;
S2: b = y;
S1: a = %b + 1;
-----------
  a = x + 1;
```

Output-dependence (Write-after-Write) - `statements S1` **and** `S2` **write to the same location:**

- **write by** `S2` **must happen after write by** `S1`:

```
S1: a = x;
S2: a = y;
-----------
    a = y;
```

```
S2: a = y;
S1: a = x;
-----------
    a = x;
```

Variable Renaming:

```
S2: a = y;
S1: %a = x;
-----------
    a = y;
```

OpenMP Synchronization

synchronization mechanisms - **coordinate execution of threads ensuring program correction:**

- Essentially **avoiding data races by respecting data dependencies.**

Suppose we would like to execute this program with OpenMP:

```
#pragma omp parallel default(none) shared(x,y)
{
    int tid = omp_get_thread_num();
    x[tid] = foo(...);
    y[tid] = x[tid] + x[tid + 1];
}
```

Q: can you identify a data-dependence (data race) on the code above?

OpenMP Synchronization

synchronization mechanisms - **coordinate execution of threads ensuring program correction:**

- Essentially **avoiding data races by respecting data dependencies.**

Suppose we would like to execute this program with OpenMP:

```
#pragma omp parallel default(none) shared(x,y)
{
    int tid = omp_get_thread_num();
    x[tid] = foo(...);
    y[tid] = x[tid] + x[tid + 1];
}
```

Data race to **shared variable** `x[tid + 1]`

Q: can you identify a data-dependence (race condition) on the code above?

A: thread `tid` **can not compute** `y[tid]` **until thread** `tid+1` **assigns the value of** `x[tid + 1]`**.**

OpenMP Synchronization

synchronization mechanisms - **coordinate execution of threads ensuring program correction:**

- Essentially **avoiding data races by respecting data dependencies.**

Suppose we would like to execute this program with OpenMP:

```
#pragma omp parallel default(none) shared(x,y)
{
    int tid = omp_get_thread_num();
    x[tid] = foo(...);
    y[tid] = x[tid] + x[tid + 1];
}
```

Wait for all threads!

Data race to **shared variable** `x[tid + 1]`

Q: can you identify a data-dependence (data race) on the code above?

A: thread `tid` **can not compute** `y[tid]` **until thread** `tid+1` **assigns the value of** `x[tid + 1]`**.**

Solution - make sure all threads finish the assignment to `x[tid]` **before any update to** `y[tid]`**!**

omp barrier construct

`barriers` - **define a point in code where threads halt their execution until all threads reach the same point.**

`barriers` are **implicitly declared at the end** of:

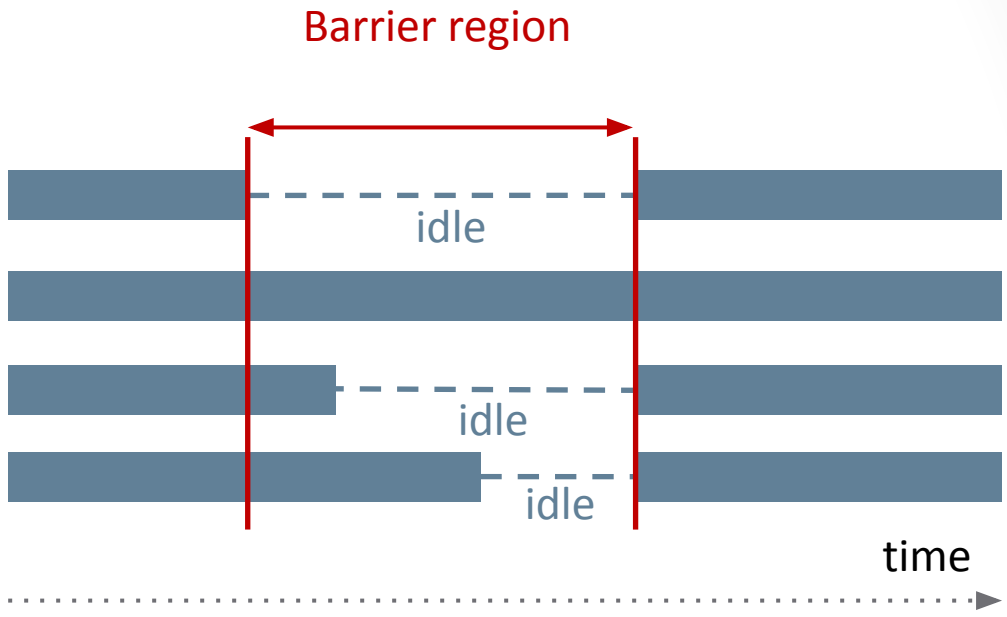
- `#omp parallel`, and work-sharing constructs.

`barriers` can be **explicitly declared** with:

- `#pragma omp barrier.`

`barriers` are **expensive** and **might limit scalability**:

- only use if necessary (**to ensure correctness**).




```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    x[tid] = foo();
    y[tid] = x[tid]+ x[tid + 1];
}
```

Data race removed!

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    x[tid] = foo();
#pragma omp barrier
    y[tid] = x[tid] + x[tid + 1];
}
```

omp critical construct

`critical` - **specifies a region of code that can only be executed by 1 thread at time (mutual exclusion).**

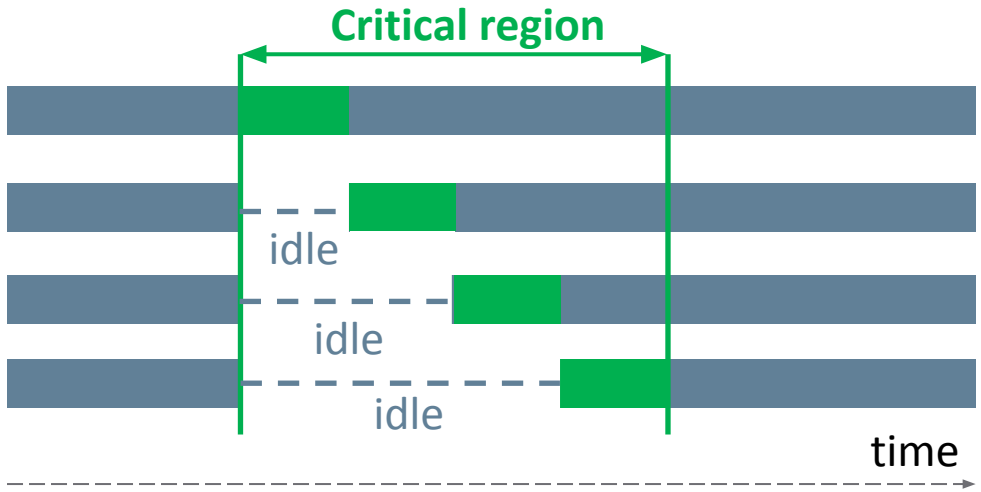
`critical` **region is defined by the :**

- `#pragma omp critical{// mutex region}`

`critical` **regions are not differentiated by OpenMP:**

- **use** `#pragma omp critical (name)` **to avoid unnecessary synchronization contention.**

Again, do not use synchronization unless necessary!




```
int result = 0;
#pragma omp parallel
{
    result += foo(...);
}
```

Data race removed!

Data race to result

```
int result = 0;
#pragma omp parallel
{
    #pragma omp critical
    result += foo(...);
}
```

omp atomic construct

`atomic` **declares that a specific memory address can only be written/read by a single thread at a time**

`atomic` **is more restrictive than** `critical`:

- **can only be applied to one instruction at a time.**
- **limited support of operations** (`+,-,*,/,&,...`).

`atomic` **generally faster than** `critical`:

- **if hardware supports low-level atomic updates.**

```
int result = 0;
#pragma omp parallel
{
  #pragma omp critical
  result += foo();
}
```

```
int result = 0;
#pragma omp parallel
{
  int tmp = foo();
  #pragma omp atomic
  result += tmp;
}
```

OpenMP Showcase: Pi v1

Q: How can we improve our parallel Pi version with synchronization constructs?

```
double partial_sum[omp_get_max_threads()] = {0.0};
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    // cyclic distribution
    for (int i=tid; i < num_steps; i+=nthreads)
    {
            double x = (i + 0.5) * step;
            partial_sum[tid] += 4.0 / (1.0 + x * x);
    }
}

double pi = 0.0;
for (int i=0; i<num_steps; i+=omp_get_max_threads())
    pi += partial_sum[tid] * step;
```

OpenMP Showcase: Pi v2

Q: How can we improve our parallel Pi version with synchronization constructs?

A: use `critical/atomic` to synchronize updates to sum (inside the parallel region).

(there is no need for the shared array now)

```
double sum = 0.0; //sum is a shared variable
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    // cyclic distribution
    for (int i=tid; i < num_steps; i+=nthreads)
    {
            double x = (i + 0.5) * step;
            #pragma omp critical
            sum += 4.0 / (1.0 + x * x);
    }
}
double pi = sum * step;
```

OpenMP Showcase: Pi v2

Q: How can we improve our parallel Pi version with synchronization constructs?

A: use `critical/atomic` to synchronize updates to sum (inside the parallel region).

(there is no need for the shared array now)

Issue - Synchronization contention

```
double sum = 0.0; //sum is a shared variable
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    // cyclic distribution
    for (int i=tid; i < num_steps; i+=nthreads)
    {
            double x = (i + 0.5) * step;
            #pragma omp critical
            sum += 4.0 / (1.0 + x * x);
    }
}
double pi = sum * step;
```

OpenMP Showcase: Pi v3

Q: How can we improve our parallel Pi version with synchronization constructs?

A: use `critical/atomic` to synchronize updates to sum (inside the parallel region).

(there is no need for the shared array now)

Issue - Synchronization contention

Solution - accumulate partial sums in private variable `sum` to avoid overhead!

```
double pi = 0.0;
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    double sum = 0.0; //sum is now private!

    // cyclic distribution
    for (int i=tid; i < num_steps; i+=nthreads)
    {
        double x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x);
    }
    #pragma omp critical
    pi += sum * step;
}
```

OpenMP Work-sharing Constructs

OpenMP ease data-parallelism via work-sharing constructs:

- `#pragma omp section/sections:`
  - Defines independent sections of code that can be assigned to a thread.

- `#pragma omp for:`
  - Distributes the iterations of a parallel loop among the team of threads.

- `#pragma task:`
  - “Dynamic” version of sections that allows more complex behaviour.
  - (not taught in this course.)

All OpenMP work-sharing constructs must:

- Be inside the dynamical extent of a parallel region (otherwise sequential behaviour).
- Be found by all threads of the team, or no thread at all.
- Have an implicit barrier at the end of its *structured-block* (unless removed).

OpenMP Sections/Section

`sections` **is a non-iterative worksharing construct that contains a set of independent *structured blocks* (`section`) to be distributed and executed by a team of threads.**

Each `section` is assigned to a single thread:

- **1 thread might execute more than 1 `section`.**

The order by which different `section` are executed is determined by runtime system:

- **order of execution is non-deterministic!**

```
#pragma omp parallel
{
    int TID = omp_get_thread_num();

    #pragma omp sections
    {
        #pragma omp section {printf("section 1\t tid:%d\n",TID);}

        #pragma omp section {printf("section 2\t tid:%d\n",TID);}

        #pragma omp section {printf("section 3\t tid:%d\n",TID);}

        #pragma omp section {printf("section 4\t tid:%d\n",TID);}
    }
}
```

```
[a12110495@alma02 ~]$ OMP_NUM_THREADS=4 ./a.out
section 1       tid:0
section 4       tid:3
section 3       tid:1
section 2       tid:2
[a12110495@alma02 ~]$ OMP_NUM_THREADS=2 ./a.out
section 1       tid:0
section 3       tid:0
section 4       tid:0
section 2       tid:1
```

OpenMP Section/Sections Exercise

Assume we want to process functions f(), g(), and x() in parallel with OpenMP sections/section:

- f(), g(), and x() take approximately the same time to execute and have no data dependencies:

```
#pragma omp parallel num_threads(2)
{
    #pragma omp sections
    {
        #pragma omp section { f();}

        #pragma omp section { g();}

        #pragma omp section { x();}
    }
}
```

Q: Can you identify a performance issue (assuming our machine only has 2 cores)?

OpenMP Section/Sections Exercise

Assume we want to process functions f(), g(), and x() in parallel with OpenMP sections/section:

- f(), g(), and x() take approximately the same time to execute and have no data dependencies:

```
#pragma omp parallel num_threads(2)
{
    #pragma omp sections
    {
        #pragma omp section { f();}

        #pragma omp section { g();}

        #pragma omp section { x();}
    }
}
```

Q: Can you identify a performance issue (assuming our machine only has 2 cores)?

A: Poor load-balancing, one of the threads will idle during the execution of the last section.

OpenMP for construct

`for` **construct automatically distributes iterations of for-loop among the members of a team of threads:**

- **number of iterations is known.**
- **no** `return`, `exit` **or** `break` **statements.**
- **index increment must be constant.**

Loop index `i` **is automatically made private:**

- **otherwise obvious data race.**

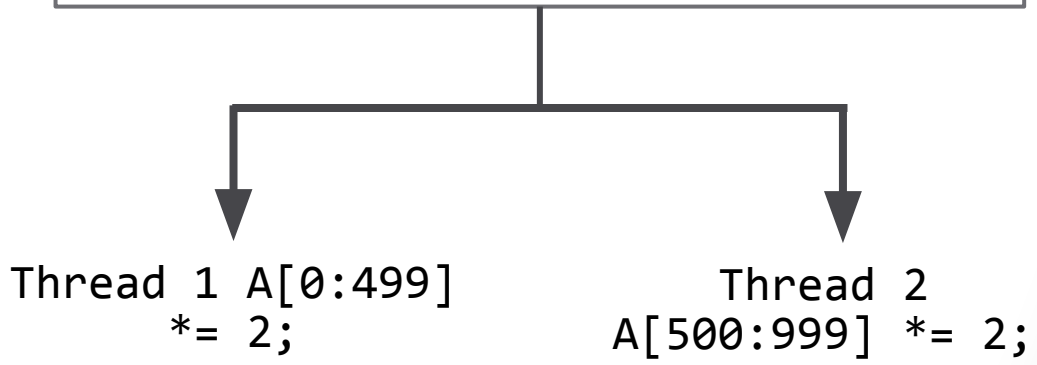
Loop must be free of loop-carried dependencies:

- **otherwise wrong result if parallel.**

`for` **and** `parallel` **constructs can be combined:**

- `#pragma omp parallel for`

```
#pragma omp parallel num_threads(2)
{
    #pragma omp for
    for (int i = 0; i < 1000; i++)
    {
        A[i]*=2;
    } // <-- implicit barrier!
}
```



OpenMP for construct

#pragma omp for can only be applied C/C++ `for`-loops:

- **OpenMP does not offer support for parallel `while`-loops.**

To parallelize a while-loop with OpenMP we need to:

- **convert the `while`-loop into a "simple" `for`-loop free of loop-carried dependences!**
  - **Removing loop-carried dependences is not always possible!**

Example procedure:

1. Convert `while`-loop → `for`-loop

```
int j, A[N];
j = 5;
int i = 0;
while (i<N)
{
  j += 2;
  A[i] = foo(j);
  i++;
}
```

2.Remove loop-carried depend:

```
int j, A[N];
j = 5;
for (int i=0; i<N; i++)
{
  j += 2;
  A[i] = foo(j);
}
```

3. Apply `parallel for` construct:

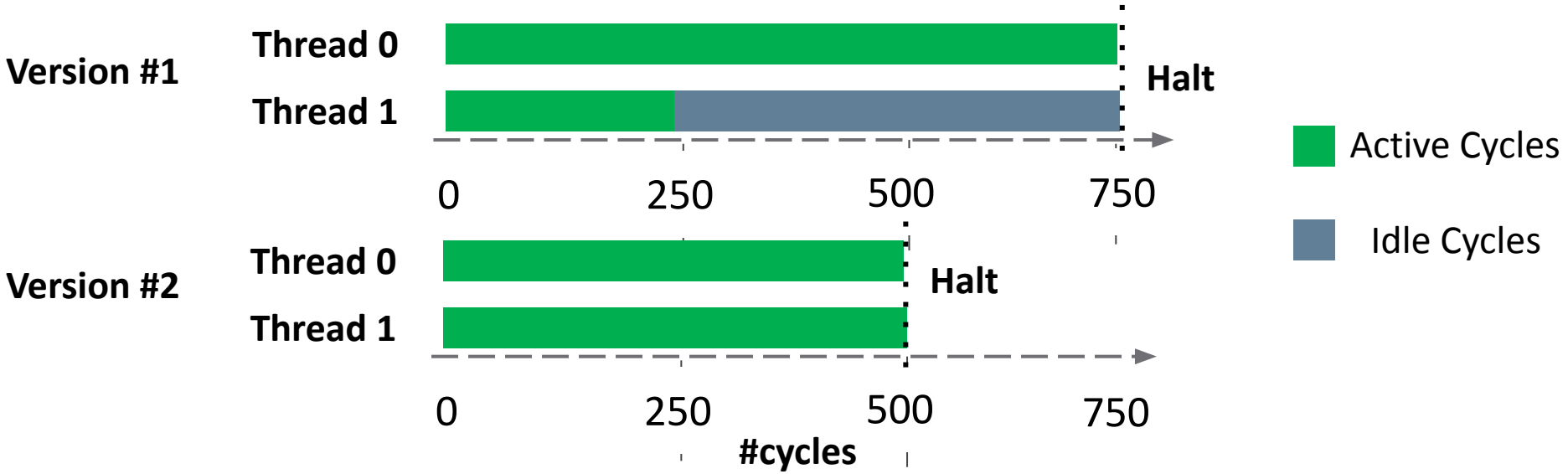
```
int A[N];
#pragma omp parallel for
for (i=0; i<N; i++) {
  int j = 5 + 2*(i+1);
  A[i] = foo(j);
}
```

Load Balancing

Load balancing is one of the most important performance aspects of parallel computing:

- **goal - minimize overall thread idle time to increase program p scalability.**

E.g. - assume a serial program which takes 1000 CPU cycles and we want to execute it in parallel:



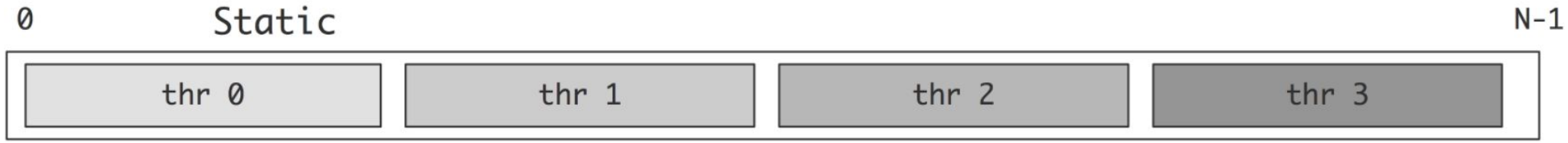
Version #1: Thread 0 - 750 cycles and Thread 1 - 250 cycles → speedup=1.3x.

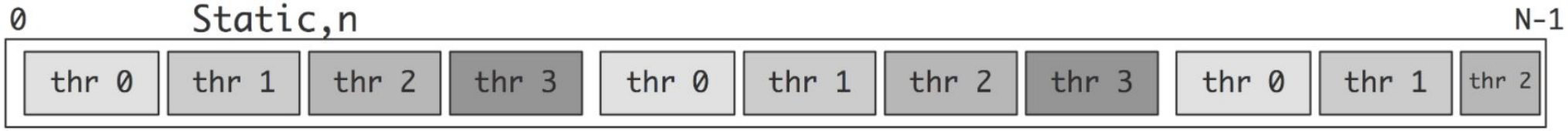
Version #2: Thread 0 - 500 cycles and Thread 1 - 500 cycles → speedup=2x.

omp for scheduling

OpenMP provides means to improve load balancing

`schedule(static [, chunk])`
Deal-out blocks of iterations of size “chunk” to each thread





`schedule(dynamic [, chunk])`
Each thread grabs “chunk” iteration off a queue until all iterations have been handled

omp for scheduling

OpenMP provides means to improve load balancing

`schedule(guided [, chunk])`
Threads dynamically grab blocks of iterations. The size of the block starts large and shrinks down to size “chunk” as the calculation proceeds

`schedule(auto)`
Schedule is left up to the runtime to choose (not necessarily any of the above)

`schedule(runtime)`
Schedule and chunk size taken from the OMP_SCHEDULE environment variable

OpenMP Showcase: Pi v3

Q: How can we improve our parallel Pi implementation with the `for` construct?

```
double pi = 0.0;

#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int nthreads = omp_get_num_threads();

    //sum becomes a private scalar
    double sum = 0.0;

    // cyclic distribution
    for (int i=tid; i < num_steps; i += nthreads)
    {
            double x = (i+0.5) * step;
            sum += 4.0 / (1.0 + x * x);
    }
    #pragma omp critical
    pi += sum * step;
}
```

OpenMP Showcase: Pi v4

Q: How can we improve our parallel Pi implementation with the `for` construct?

1. replace "our" parallel for with:

```
#pragma omp for
for (int i=0; i<num_steps; i++)
{
    double x = (i+0.5) * step;
    sum += 4.0 / (1.0 + x * x);
}
```

2. remove unnecessary routines:

```
int tid = omp_get_thread_num();
int nthreads = omp_get_num_threads();
```

```
int main(int argc, char* argv[])
{
    ...
    double pi = 0.0;

    #pragma omp parallel
    {
        double sum = 0.0;

        #pragma omp for
        for (int i = 0; i < num_steps; i++)
        {
            double x = (i+0.5) * step;
            sum += 4.0 / (1.0 + x * x);
        }

        #pragma omp critical
        pi += sum * step;
    }
    ...
}
```

omp for nowait clause

`nowait` - **removes the implicit** `barrier` **found at the end of a worksharing construct structured-block :**

- **can be applied to** `for`, `sections`, **...**

In some situations implicit `barriers` **are not needed to ensure the correction of our solution:**

- **removing it reduces synchronization overhead,**
- **and minimize thread idle time.**

In this snippet the first and second loop have no data dependencies, therefore:

- **If thread finishes first loop, it can start second loop**

```
#pragma omp parallel private(x,y)
{
  ...
  #pragma omp for nowait
  for (int i=0; i < len1; i++)
  {
    x[i]++;
  } // <- implicit barrier removed

  #pragma omp for
  for (int i=0; i < len2; i++)
  {
    y[i] /= 2;
  }
}
```

omp reduction clause

`reduction (op: list)`

Inside a parallel or a work-sharing construct

- A local copy of each list variable is made and initialized depending on the “op” (e.g. 0 for “+”);
- Updates occur on the local copy
- Local copies are reduced into a single value and combined with the original global value

Variables in the “list” must be shared in the enclosing parallel region

---

```
int sum=0, A[N];
#pragma omp parallel{
    int sum_private = 0;

    #pragma omp for
    for (int i=0; i < N; i++)
        sum_private += A[i];

    #pragma omp atomic
    sum += sum_private
}
```

Semantically Equivalent

```
int sum, A[N];
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++)
  sum += A[i];
```

OpenMP Showcase: Pi v4

Q: How can we improve our parallel Pi implementation with a `reduction` clause?

```
int main(int argc, char* argv[])
{
    ...
    double pi = 0.0;

    #pragma omp parallel
    {
        double sum = 0.0; // sum is private

        #pragma omp for
        for (int i = 0; i < num_steps; i++)
        {
            double x = (i + 0.5) * step;
            sum += 4.0 / (1.0 + x * x);
        }

        #pragma omp critical
        pi += sum * step;
    }
    ...
}
```

OpenMP Showcase: Pi v5

Q: How can we improve our parallel Pi implementation with a `reduction` clause?

A: Just replace the "manual reduction":

1. **make `sum` shared.**
2. **modify for construct:**
   `#pragma omp for reduction(+:sum)`
3. **Access shared `sum` after parallel region!**

```
int main(int argc, char* argv[])
{
    ...
    double pi = 0.0;
    double sum;
    #pragma omp parallel
    {
        #pragma omp for reduction(+:sum)
        for (int i = 0; i < num_steps; i++)
        {
                double x = (i + 0.5) * step;
                sum += 4.0 / (1.0 + x * x);
        }
    }

    pi += sum * step;
    ...
}
```

OpenMP master Construct

`master` - **construct equivalent to:**

- `if(omp_get_thread_num() == 0) {...}`

`master` **is not a work-sharing construct:**

- **Its structured-block does not contain an implicit barrier.**

`master` **is typically used for:**

- **debugging purposes**
- **Initialize shared data structures ← (be mindful of the missing barrier!)**

```
#pragma omp parallel
{
    #pragma omp master
    {
        // do something that only one thread should do!
    } // <-- no implicit barrier
}
```

OpenMP single construct

The single construct denotes a block of code that is executed by only one thread

A barrier is implied at the end of the single block (unless there is a nowait clause)

```
#pragma omp parallel
{
        …
        #pragma omp single
        {
                // code executed by a single thread
        }
        …
}
```

Implicit barrier!

The Last Slide

Questions?